

GUÍA DE EJERCICIOS N.º 3

Modularización y contratos

Carrera: Ingeniería en Informática e Ingeniería en Telecomunicaciones

Materia: Programación II

Alumno: Santiago Salvay

Profesora: Nano, Mónica Liliana

Semilla: S = 15

Compilador: Microsoft Visual Studio Community



UNIVERSIDAD
Blas Pascal

Índice

Semilla personal.....	3
Ejercicio 1 · Radiografía de un main monolítico.....	3
Ejercicio 2 · Laboratorio de paso de parámetros.....	5
Ejercicio 3 · Tres archivos y dos clases de error.....	9
Ejercicio 4 · Pruebas antes de confiar.....	14
Ejercicio 5 · Mini-proyecto: monitor de mediciones.....	15
Anexo de IA.....	20

Semilla personal

D = 46

S = 15 (D = 46 $\rightarrow$ S = 10 + (46 mod 41) = 10 + 5 = 15).

Ejercicio 1 · Radiografía de un main monolítico

1)

Si uno mira con cuidado el main() original, en realidad está haciendo cuatro cosas distintas que no tienen nada que ver entre sí, solo están una atrás de la otra:

- Leer cuántos datos vienen (la variable n).
- Leer esos n valores y guardarlos en el vector.
- Filtrar los valores válidos (los que caen dentro de un rango) y acumular su suma y su cantidad.
- Calcular el promedio y mostrarlo, pero solo si hubo al menos un valor válido.

Como esas cuatro cosas son responsabilidades distintas, propongo separarlas en estas funciones:

1. leerCantidad() : lee n.
2. leerDatos(n) : lee los n valores y devuelve el vector.
3. enRango(x, minimo, maximo) : dice si un valor puntual es válido.
4. promedioValidos(datos, mínimo, máximo) : recorre el vector, usa enRango para filtrar y calcula el promedio de los válidos.
5. mostrarResultado(promedio, huboValidos) : imprime el resultado (esta es opcional, se podría dejar la impresión en main, pero separarla deja a main sin ningún cout de por medio).

2)

Función	Firma	Precondición	Postcondición	¿Efecto?
leerCantidad	int leerCantidad();	std::cin disponible	Devuelve el n leído; no	Si

			valida que sea positivo	
leerDatos	<code>std::vector<double> leerDatos(int n);</code>	<code>n >= 0</code>	Devuelve un vector de tamaño n con los valores leídos, en el mismo orden en que entraron	Si
enRango	<code>bool enRango(double x, double minimo, double maximo);</code>	<code>minimo <= maximo</code>	Devuelve true si <code>minimo <= x <= maximo</code>	No
promedioValidos*	<code>bool promedioValidos(const std::vector<double>& datos, double minimo, double maximo, double& promedio);</code>	<code>minimo <= maximo</code>	Si hay al menos un valor válido, devuelve true y deja el promedio en promedio; si no hay ninguno, devuelve false y no toca promedio	No
mostrarResultado	<code>void mostrarResultado(double promedio, bool huboValidos);</code>	Ninguna	No cambia nada del programa	Si

*promedioValidos devuelve bool en vez de imprimir directamente el resultado así separamos "calcular" de "mostrar", que es justo el problema que tenía el main original.

3)

En main() debería quedar solamente la coordinación: pedir n, pedir los datos, calcular el promedio y decidir qué mostrar según si hubo válidos o no. Es decir, cuatro llamadas a función y un if, nada más.

No conviene ir más allá y hacer una función por cada línea, una para "sumar", otra para "incrementar el contador", etc. Porque esas líneas sueltas no representan una idea completa por sí mismas sino solo tienen sentido dentro del recorrido que hace promedioValidos. Partirlas en más funciones no baja el acoplamiento ni te deja probar nada por separado; lo único que logra es más funciones para leer y más saltos para seguir el hilo del código.

4)

Cambio el límite superior de 100.0 a $50.0 + S$. Con $S = 15$, el nuevo rango válido queda $[0, 65]$ (antes era $[0, 100]$).

Los cuatro valores a predecir, con $S = 15$:

Valor	Cálculo	Resultado	¿Cae en $[0, 65]$?
$S - 1$	$15 - 1$	14	Si
S	-	15	Si
$S + 40$	$15 + 40$	55	SI
$S + 50$	$15 + 50$	65	Si, justo en el límite

Si esos cuatro valores son los datos que entran al programa ($n = 4$), los cuatro son válidos, entonces:

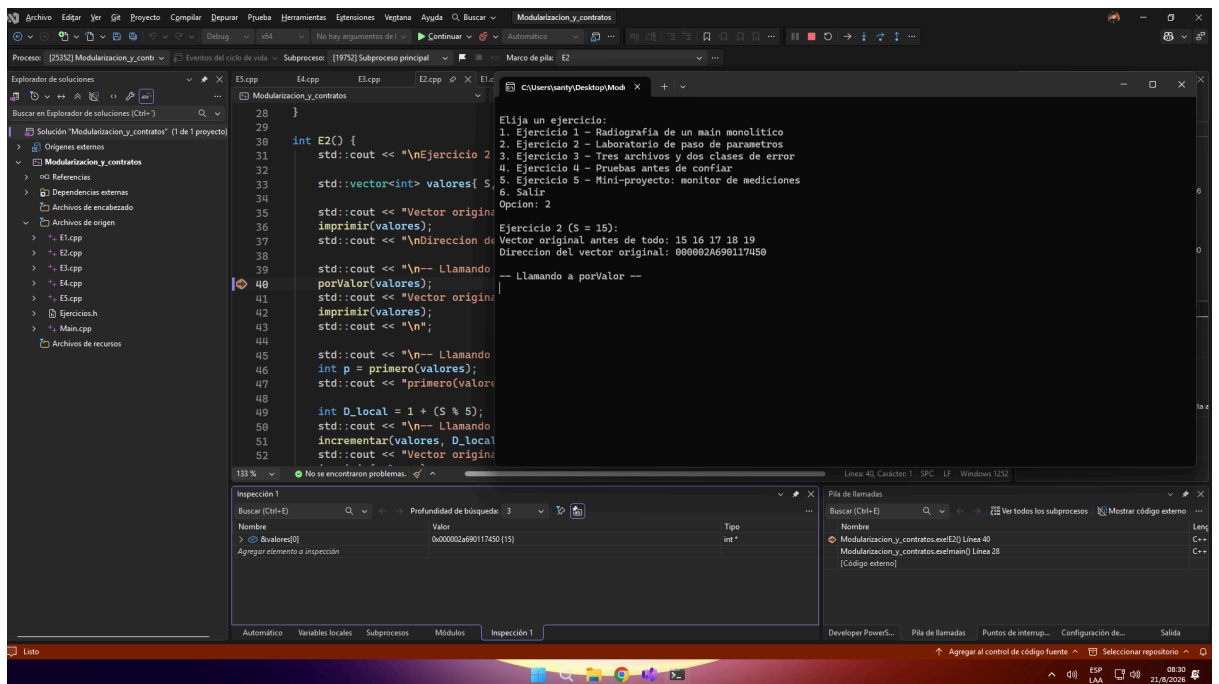
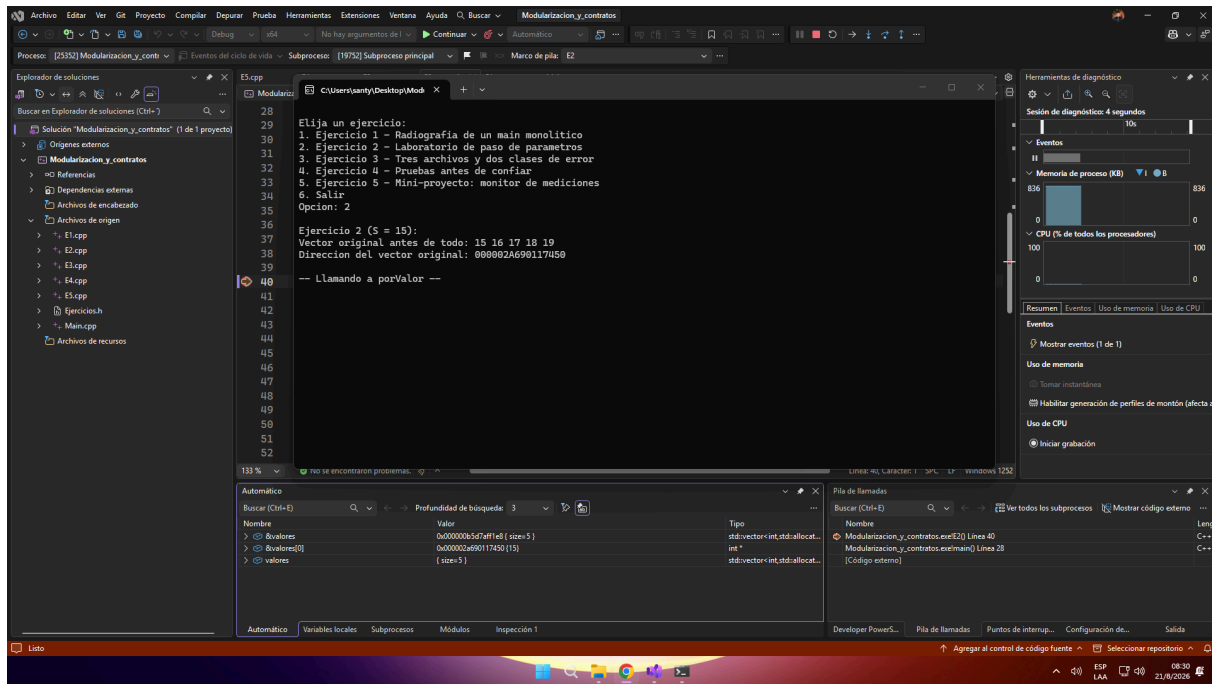
- Válidos = 4
- Suma = $14 + 15 + 55 + 65 = 149$
- Promedio = $149 / 4 = 37.25$

Lo que nos interesa de esta predicción es que el resultado no cambia respecto al límite viejo de 100.0: como ninguno de los cuatro valores supera 65, tampoco supera 100, así que el cambio de límite es invisible con este conjunto de datos.

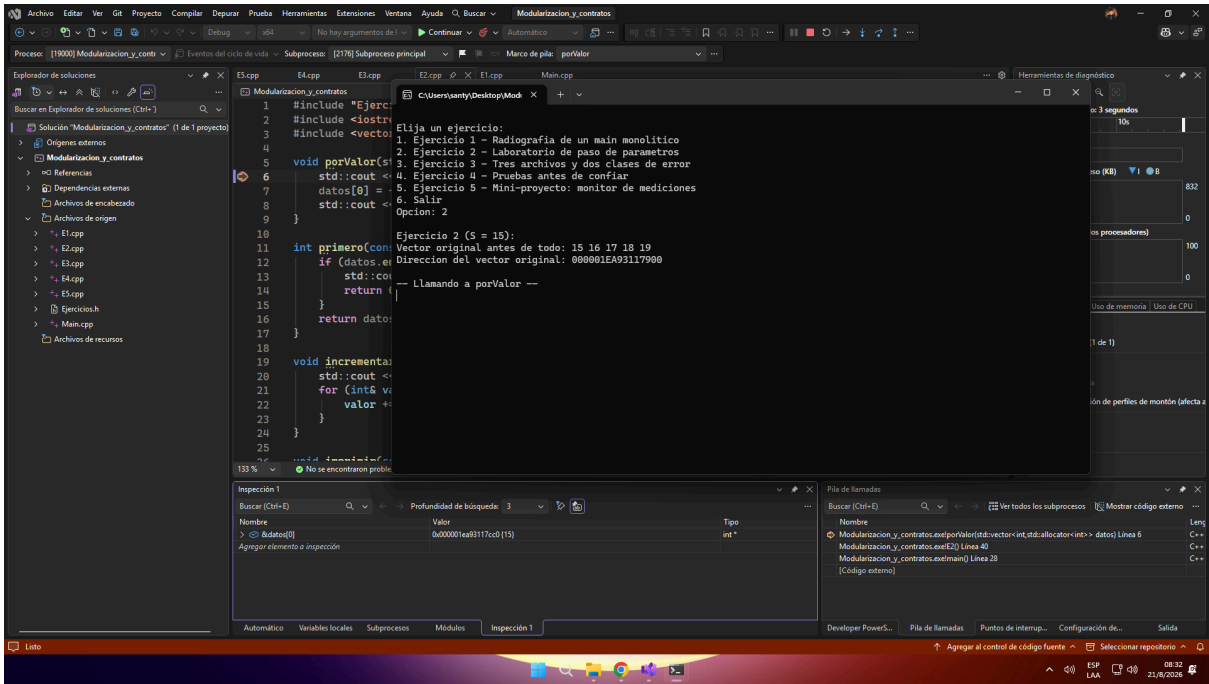
Para que se note la diferencia habría que meter un valor entre 66 y 100 (por ejemplo $S+51 = 66$), que antes era válido

Ejercicio 2 · Laboratorio de paso de parámetros

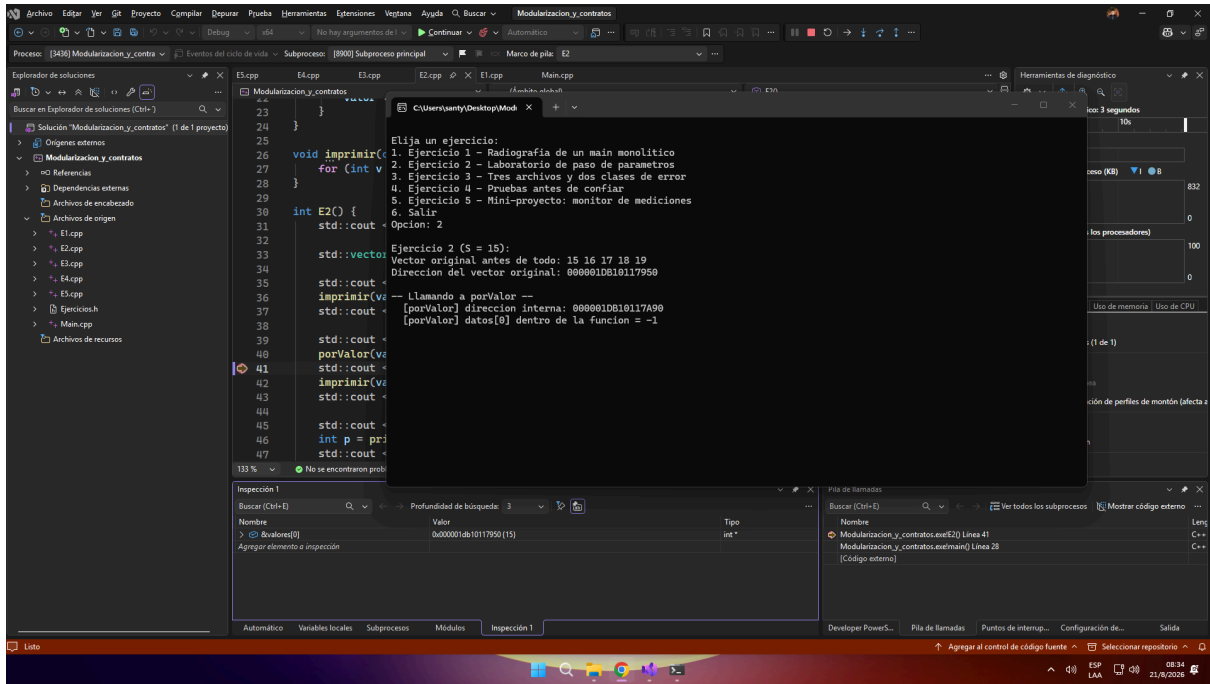
1)



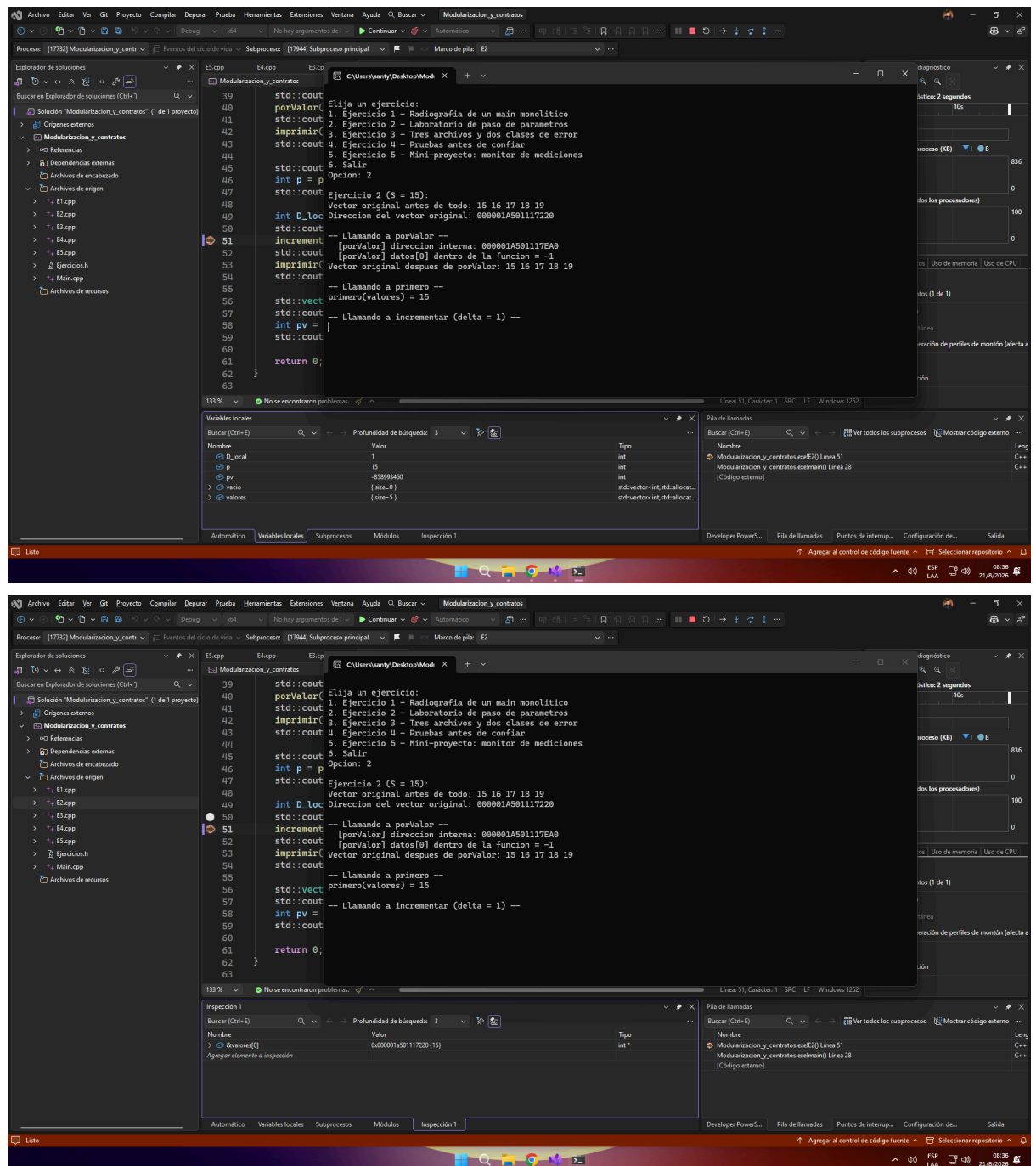
2)



3)



4)



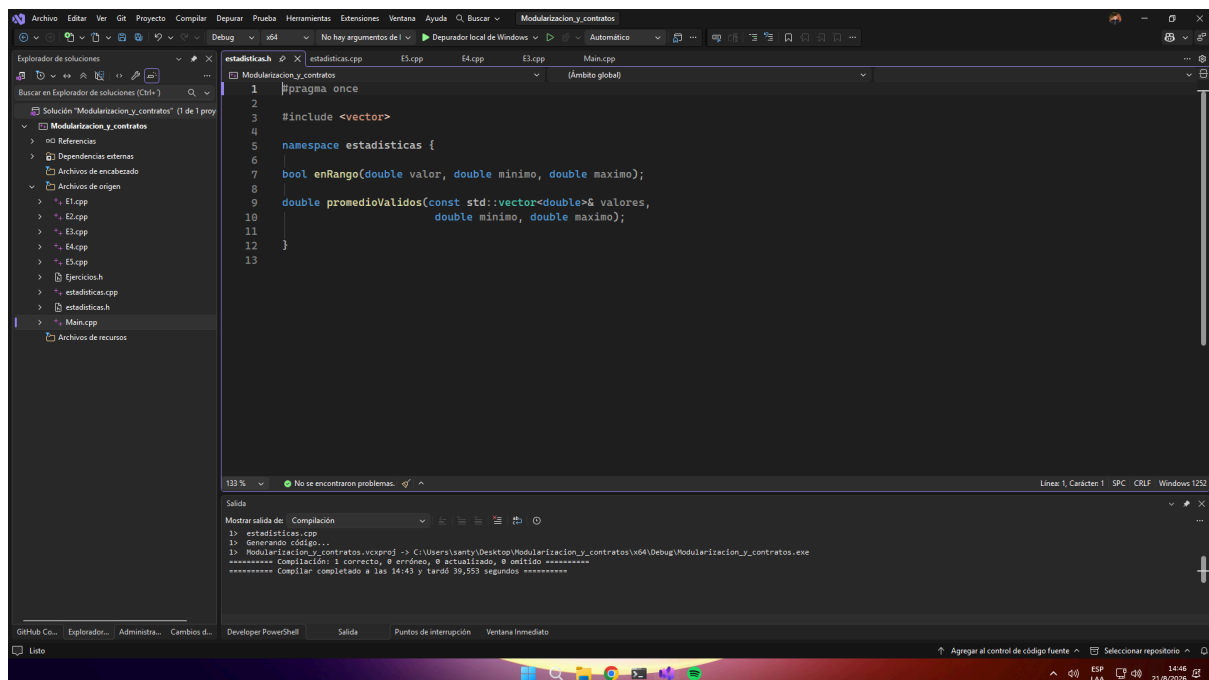
5)

Primero solo necesita mirar el vector, no tocarlo. Si lo recibiera por valor (`std::vector<int>`), el programa copiaría los cinco elementos cada vez que se llama a la función, gastando memoria y tiempo sin ninguna razón, porque después esa copia se descarta. Usando `const std::vector<int>&` le pasamos una referencia al vector original, así que no hay copia: la función trabaja directo sobre el mismo vector, pero el `const` actúa como un candado, si en algún momento me equivoco y escribo `datos[0] = 5` dentro de `primero`, el compilador me lo va a marcar como error antes de que el programa siquiera corra. Es la forma de decirle a cualquiera que lea la firma de la función "esto solo lee, te lo garantizo" sin tener que confiar en que yo (o quien la use después) me acuerde de no tocar nada.

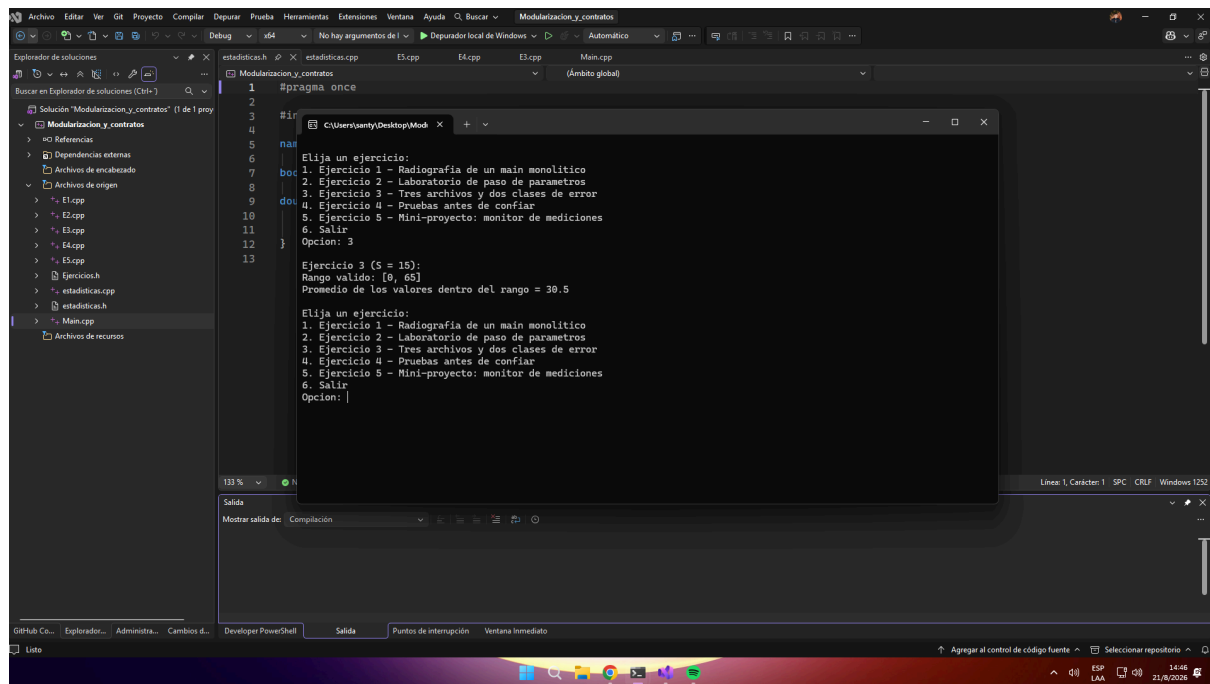
Incrementar, en cambio, tiene el propósito contrario: existe justamente para modificar los valores del vector que le pasan. Ahí `std::vector<int>&` (sin `const`) es lo correcto porque necesito que datos sea el mismo vector que tiene quien llama a la función, no una copia, si fuera por valor, yo sumaría el delta a una copia que se destruye al terminar la función, y el vector original en `main/E2` quedaría exactamente igual, lo cual rompería el propósito de la función. La referencia no constante me da acceso de lectura y escritura a ese mismo objeto en memoria (por eso en el depurador la dirección de datos adentro de `incrementar` es idéntica a la de valores afuera), y quitar el `const` es coherente porque estoy siendo honesto en la firma: esta función va a cambiar lo que le pasaste, prepárate para eso.

Ejercicio 3 · Tres archivos y dos clases de error

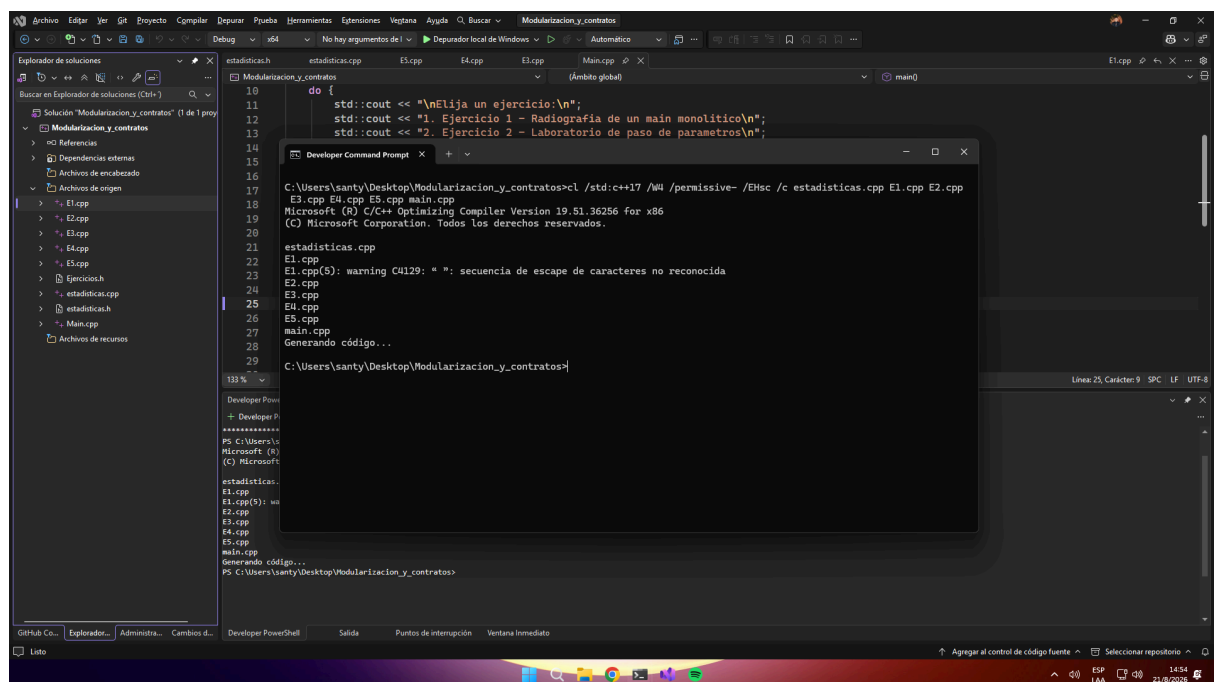
1)



2)



3)



```

10 do {
11     std::cout << "\nElija un ejercicio:\n";
12     std::cout << "1. Ejercicio 1 - Radiografía de un main monolitico\n";
13     std::cout << "2. Ejercicio 2 - Laboratorio de paso de parametros\n";
14
15
16
17
18
19 Elija un ejercicio:
20 1. Ejercicio 1 - Radiografía de un main monolitico
21 2. Ejercicio 2 - Laboratorio de paso de parametros
22 3. Ejercicio 3 - Tres archivos y dos clases de error
23 4. Ejercicio 4 - Pruebas antes de confiar
24 5. Ejercicio 5 - Mini-proyecto: monitor de mediciones
25 6. Salir
26 Opcion: 3
27
28 Ejercicio 3 (S = 15):
29 Rango valido: [0, 65]
30 Promedio de los valores dentro del rango = 30.5
31
32 Elija un ejercicio:
33 1. Ejercicio 1 - Radiografía de un main monolitico
34 2. Ejercicio 2 - Laboratorio de paso de parametros
35 3. Ejercicio 3 - Tres archivos y dos clases de error
36 4. Ejercicio 4 - Pruebas antes de confiar
37 5. Ejercicio 5 - Mini-proyecto: monitor de mediciones
38 6. Salir
39 Opcion: 6
40 Saliendo...
41
42 estadisticas:
43 E1.cpp
44 E1.cpp(S): w
45 E2.cpp
46 E3.cpp
47 E4.cpp
48 E5.cpp
49 main.cpp
50 Generando código...
51 PS C:\Users\santy\Desktop\Modularizacion_y_contratos>

```

```

C:\Users\santy\Desktop\Modularizacion_y_contratos>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 3CDC-184A

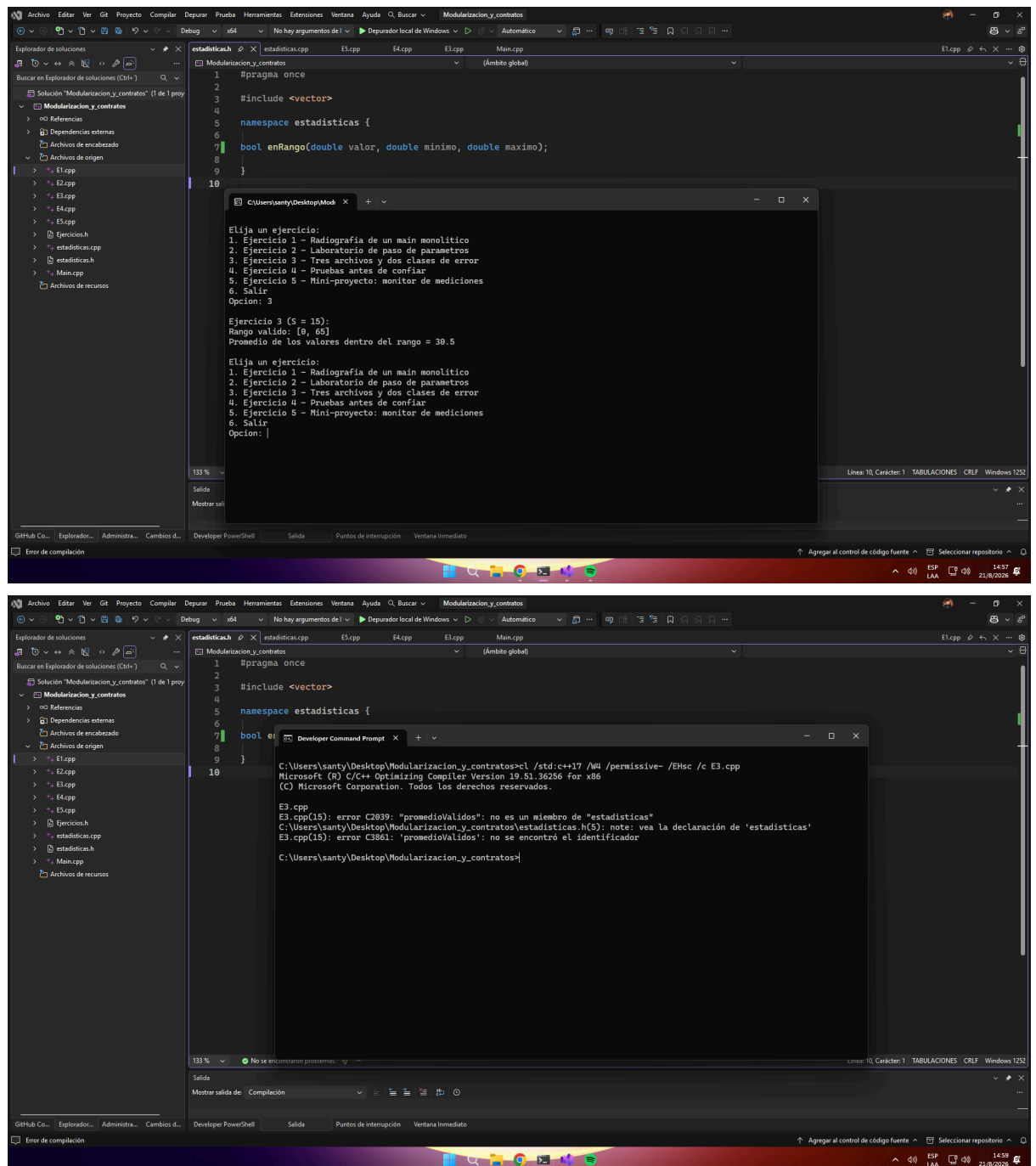
Directorio de C:\Users\santy\Desktop\Modularizacion_y_contratos

21/08/2026 14:54 <DIR> .
21/08/2026 14:19 <DIR> ..
21/08/2026 14:55 <DIR> 269.824 demo.exe
20/08/2026 20:09 162 E1.cpp
21/08/2026 14:54 63.114 E1.obj
21/08/2026 08:21 1.807 E2.cpp
21/08/2026 14:54 169.654 E2.obj
21/08/2026 14:41 572 E3.cpp
21/08/2026 14:54 158.334 E3.obj
20/08/2026 19:45 152 E4.cpp
21/08/2026 14:54 142.479 E4.obj
20/08/2026 19:45 152 E5.cpp
21/08/2026 14:54 142.479 E5.obj
21/08/2026 19:45 85 Ejercicios.h
21/08/2026 14:41 557 estadisticas.cpp
21/08/2026 14:42 247 estadisticas.h
21/08/2026 14:54 2.236 estadisticas.obj
20/08/2026 19:17 382.346 Guia_Clase_1_Modularizacion_Cpp.pdf
20/08/2026 20:09 1.353 main.cpp
21/08/2026 14:54 166.201 main.obj
20/08/2026 19:38 <DIR> Modulari.92754475
21/08/2026 08:20 223 Modularizacion_y_contratos.slnx
21/08/2026 14:42 7.312 Modularizacion_y_contratos.vcxproj
21/08/2026 14:41 1.941 Modularizacion_y_contratos.vcxproj.filters
20/08/2026 19:18 168 Modularizacion_y_contratos.vcxproj.user
20/08/2026 19:46 <DIR> x64
20/08/2026 19:46 22 archivos 1.511.398 bytes
20/08/2026 19:46 4 dirs 675.774.169.896 bytes Libres

C:\Users\santy\Desktop\Modularizacion_y_contratos>

```

4)



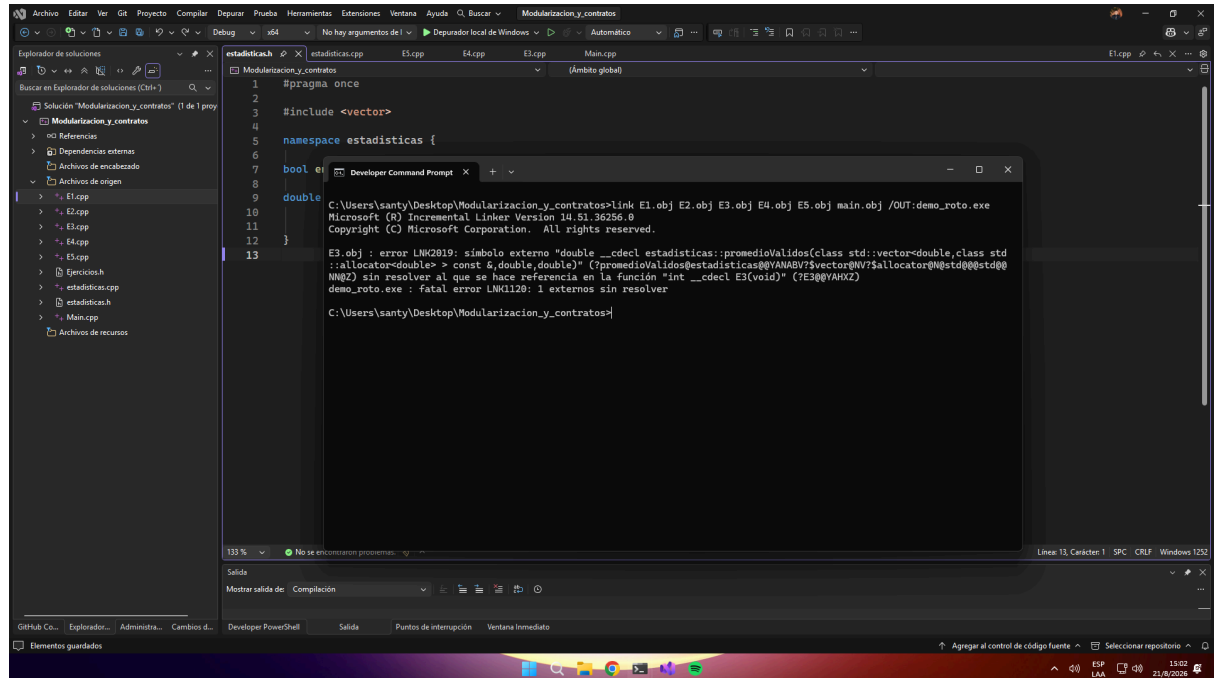
Para provocar este error fui a estadisticas.h y borré la línea de promedioValidos, dejando solo enRango declarada. Compilé E3.cpp de nuevo y esto fue lo que tiré:

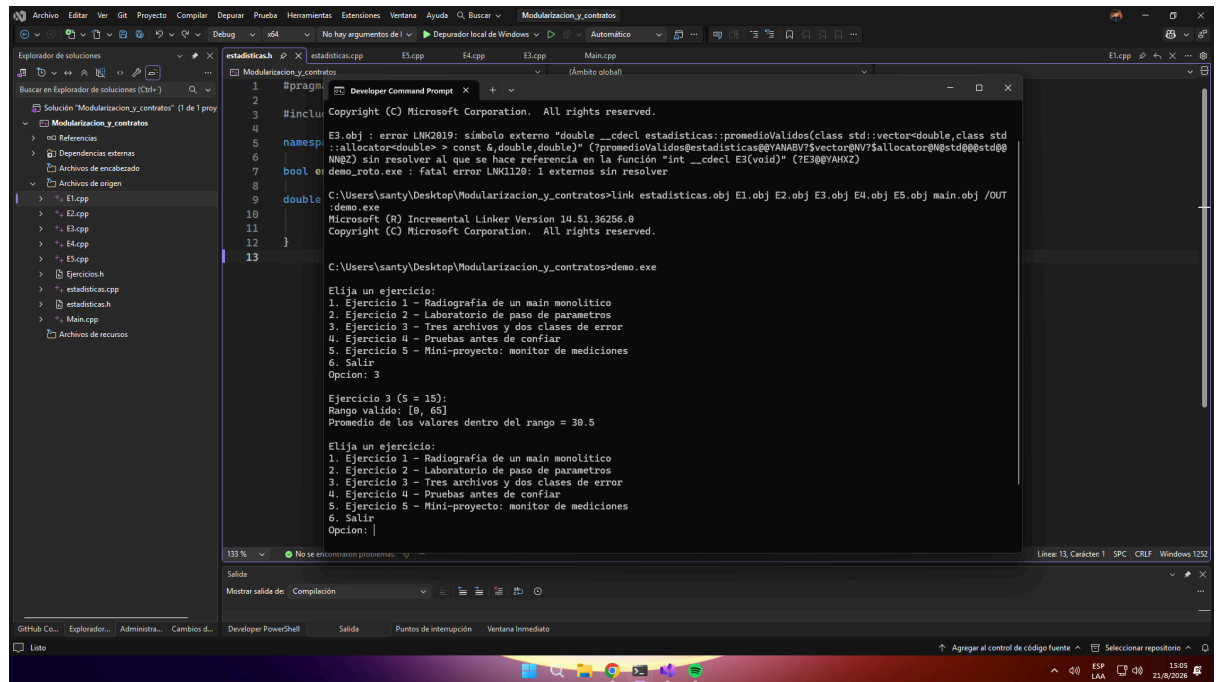
E3.cpp(15): error C2039: 'promedioValidos': no es un miembro de 'estadisticas'
 estadisticas.h(5): note: vea la declaración de 'estadisticas'
 E3.cpp(15): error C2065: 'promedioValidos': no se encontró el identificador

Lo que pasa acá es que el compilador va archivo por archivo, y lo único que "sabe" de otro módulo es lo que ve en el header incluido. Cómo saqué la declaración, cuando el compilador llega a la línea donde llamo a estadisticas::promedioValidos no tiene ni idea de que esa función existe para él, directamente no está. No es que no

encuentre el .cpp ni el .obj, es que nunca le mostré el prototipo, así que ni siquiera llega a intentar compilar la llamada bien. Lo arreglé volviendo a poner la declaración en el header, que es lo que tendría que haber estado desde un principio.

5)





Con la declaración ya restaurada, esta vez enlacé sin incluir estadisticas.obj en el comando final. El programa compiló cada archivo sin problema, pero al enlazar tiré esto:

E3.obj : error LNK2019: símbolo externo "double __cdecl estadisticas::promedioValidos(...)" sin resolver al que se hace referencia en la función "int __cdecl E3(void)" (7E3B@YAHKZ)
demo_roto.exe : fatal error LNK1120: 1 externos sin resolver

Acá la diferencia con el error anterior es que este no lo tira el compilador, lo tira el linker. E3.cpp compiló bien porque el header sí tenía la declaración de promedioValidos, entonces el compilador confía en que en algún lado va a estar el código real de esa función y genera la llamada como si nada. El problema aparece recién cuando el linker tiene que juntar todos los .obj en un .exe: busca dónde está definida esa función y no la encuentra, porque justo el archivo que la tiene (estadisticas.obj) no se lo pasé. Agregando ese .obj al comando de enlace, el símbolo aparece y el programa corre normal. La diferencia con el punto 4 es justamente esa: antes el compilador ni sabía que la función existía, y ahora sí sabe que existe pero no encuentra dónde vive.

Ejercicio 4 · Pruebas antes de confiar

1) Tabla de predicciones:

Caso	Entrada	Predicción	Observado	Conclusión
------	---------	------------	-----------	------------

Normal	{10,20,30}	20	20	Coincide
Mínimo	0	true	true	Coincide
Máximo	65 (50 + S)	true	true	Coincide
Fuera inferior	-1	false	false	Coincide
Fuera superior	66 (51 + S)	false	false	Coincide
Sin válidos	{-5, -10,165}	std::invalid_argument	std::invalid_argument	Coincide

```

1 #include "Ejercicios.h"
2 #include "estadisticas.h"
3 #include <iostream>
4 #include <string>
5 #include <vector>
6
7 int E4() {
8     // Elige un ejercicio:
9     const std::vector<std::string> opciones = {
10         "1. Ejercicio 1 - Radiografía de un main monolítico",
11         "2. Ejercicio 2 - Laboratorio de paso de parámetros",
12         "3. Ejercicio 3 - Tres archivos y dos clases de error",
13         "4. Ejercicio 4 - Pruebas antes de confiar",
14         "5. Ejercicio 5 - Mini-proyecto: monitor de mediciones",
15         "6. Salir"
16     };
17     std::cout << "Opciones: ";
18     for (const auto &opcion : opciones) {
19         std::cout << opcion << " ";
20     }
21     std::cout << "\n";
22
23     // Elige un ejercicio:
24     bool validado = false;
25     while (!validado) {
26         std::string opcion;
27         std::cout << "Elige un ejercicio: ";
28         std::getline(std::cin, opcion);
29
30         if (opcion == "1") {
31             // Ejercicio 1 - Radiografía de un main monolítico
32             std::cout << "Normal: predijo 20, dio 20\n";
33             validado = true;
34         } else if (opcion == "2") {
35             // Ejercicio 2 - Laboratorio de paso de parámetros
36             std::cout << "Mínimo (0): predijo true, dio 1\n";
37             validado = true;
38         } else if (opcion == "3") {
39             // Ejercicio 3 - Tres archivos y dos clases de error
40             std::cout << "Máximo (65): predijo true, dio 1\n";
41             validado = true;
42         } else if (opcion == "4") {
43             // Ejercicio 4 - Pruebas antes de confiar
44             std::cout << "Fuera por abajo (-1): predijo false, dio 0\n";
45             validado = true;
46         } else if (opcion == "5") {
47             // Ejercicio 5 - Mini-proyecto: monitor de mediciones
48             std::cout << "Fuera por arriba (66): predijo false, dio 0\n";
49             validado = true;
50         } else if (opcion == "6") {
51             // Salir
52             std::cout << "Sin válidos: predijo excepción, tiro invalid_argument (promedioValidos: ningún valor cae dentro del rango)\n";
53             validado = true;
54         } else {
55             std::cout << "Opción no válida. Intenta de nuevo.\n";
56         }
57     }
58 }
59
60 int main() {
61     E4();
62     return 0;
63 }

```

Ejercicio 5 · Mini-proyecto: monitor de mediciones

1)

```

1  #include "Ejercicios.h"
2  #include "mediciones.h"
3  #include <iostream>
4  #include <vector>
5  #include <stdexcept>
6
7  namespace {
8
9  void mostrarInvalidas(const std::vector<double>& valores, double minimo, double maximo) {
10     std::cout << "Descartadas (fuera de [" << minimo << ", " << maximo << "]): ";
11     bool alguna = false;
12     for (double valor : valores) {
13         if (!mediciones::esValida(valor, minimo, maximo)) {
14             std::cout << valor << ' ';
15             alguna = true;
16         }
17     }
18     if (!alguna) std::cout << "ninguna";
19     std::cout << "\n";
20 }
21
22 // Reune las cuatro funciones del modulo mediciones para producir el resumen de un lote.
23 void mostrarResumen(const std::vector<double>& valores, double minimo, double maximo) {
24     mostrarInvalidas(valores, minimo, maximo);
25 }

```

118 % No se encontraron problemas. Línea: 71, Carácter: 1 SPC LF UTF-8

Salida
Developer PowerShell

Listo

Agregar al control de código fuente Seleccionar repositorio

09:09 26/8/2026

```

1  #include "mediciones.h"
2  #include <stdexcept>
3
4  namespace mediciones {
5
6  bool esValida(double valor, double minimo, double maximo) {
7      return valor >= minimo && valor <= maximo;
8  }
9
10 double promedio(const std::vector<double>& valores, double minimo, double maximo) {
11     double suma = 0.0;
12     int cantidad = 0;
13
14     for (double valor : valores) {
15         if (esValida(valor, minimo, maximo)) {
16             suma += valor;
17             ++cantidad;
18         }
19     }
20
21     if (cantidad == 0) {
22         throw std::invalid_argument("promedio: ninguna medicion cae dentro del rango");
23     }
24
25     return suma / cantidad;

```

118 % No se encontraron problemas. Línea: 1, Carácter: 1 SPC LF UTF-8

Salida
Developer PowerShell

Listo

Agregar al control de código fuente Seleccionar repositorio

09:10 26/8/2026

```

1 #pragma once
2
3 #include <vector>
4
5 namespace mediciones {
6
7     bool esValida(double valor, double minimo, double maximo);
8
9     // Lanza std::invalid_argument si ninguna medicion cae en [minimo, maximo].
10    double promedio(const std::vector<double>& valores, double minimo, double maximo);
11
12    int contarValidas(const std::vector<double>& valores, double minimo, double maximo);
13
14    // Lanza std::invalid_argument si ninguna medicion cae en [minimo, maximo].
15    double minimoValida(const std::vector<double>& valores, double minimo, double maximo);
16
17    // Lanza std::invalid_argument si ninguna medicion cae en [minimo, maximo].
18    double maximoValida(const std::vector<double>& valores, double minimo, double maximo);
19
20 }
21

```

118 % No se encontraron problemas. Líneas: 21, Carácter: 1 TABULACIONES LF UTF-8

Salida
Developer PowerShell Salida

Listo

Agregar al control de código fuente Seleccionar repositorio

09:10 26/8/2026

```

1 #pragma
2 2. Ejercicio 2 - Laboratorio de paso de parametros
3 3. Ejercicio 3 - Tres archivos y dos clases de error
4 4. Ejercicio 4 - Pruebas antes de confiar
5 5. Ejercicio 5 - Mini-proyecto: monitor de mediciones
6 6. Salir
7 Opcion: 5
8
9 bool Ejercicio 5 (S = 15), rango valido [-15, 30]
10
11 // La
12 double Descartadas (fuera de [-15, 30]): -20 40
13 Validas: 6 de 8
14 Promedio: 6.33333
15 int c
16 Minimo valido: -15
17 // La
18 double Maximo valido: 30
19
20 double Escenario 1
21 Descartadas (fuera de [-15, 30]): -16 -17 31 80
22 Validas: 0 de 4
23 Sin mediciones validas: promedio: ninguna medicion cae dentro del rango
24
25 Modificacion final
26 Nuevo rango valido [-15, 45]
27 Descartadas (fuera de [-15, 45]): -20
28 Validas: 7 de 8
29 Promedio: 11.1429
30 Minimo valido: -15
31 Maximo valido: 40
32
33 Elija un ejercicio:

```

118 % No

Salida
Developer PowerShell Salida

Compilación correcta

Agregar al control de código fuente Seleccionar repositorio

09:13 26/8/2026

2)

The screenshot shows the Visual Studio Code editor with the file 'mediciones.cpp' open. The code defines a namespace 'mediciones' and implements several functions. The 'esValida' function is defined as follows:

```

1  #pragma once
2
3  #include <vector>
4
5  namespace mediciones {
6
7      bool esValida(double valor, double minimo, double maximo);
8
9      // Lanza std::invalid_argument si ninguna medicion cae en [minimo, maximo].
10     double promedio(const std::vector<double>& valores, double minimo, double maximo);
11
12     int contarValidas(const std::vector<double>& valores, double minimo, double maximo);
13
14     // Lanza std::invalid_argument si ninguna medicion cae en [minimo, maximo].
15     double minimoValida(const std::vector<double>& valores, double minimo, double maximo);
16
17     // Lanza std::invalid_argument si ninguna medicion cae en [minimo, maximo].
18     double maximoValida(const std::vector<double>& valores, double minimo, double maximo);
19 }
20
21

```

The right sidebar shows the 'Explorador de soluciones' (Solution Explorer) with the project 'Modularizacion_y_contratos' and its files: 'E1.cpp', 'E2.cpp', 'E3.cpp', and 'E4.cpp'. The status bar at the bottom indicates 'Linea: 21, Carácter: 1' and 'TABULACIONES LF UTF-8'.

3)

The screenshot shows the Visual Studio Code editor with the file 'mediciones.cpp' open. The code implements the 'promedio' function as follows:

```

7      return valor >= minimo && valor <= maximo;
8  }
9
10     double promedio(const std::vector<double>& valores, double minimo, double maximo) {
11         double suma = 0.0;
12         int cantidad = 0;
13
14         for (double valor : valores) {
15             if (esValida(valor, minimo, maximo)) {
16                 suma += valor;
17                 ++cantidad;
18             }
19         }
20
21         if (cantidad == 0) {
22             throw std::invalid_argument("promedio: ninguna medicion cae dentro del rango");
23         }
24
25         return suma / cantidad;
26     }
27
28     int contarValidas(const std::vector<double>& valores, double minimo, double maximo) {
29         int cantidad = 0;
30         for (double valor : valores) {
31             if (esValida(valor, minimo, maximo)) {

```

The right sidebar shows the 'Explorador de soluciones' (Solution Explorer) with the project 'Modularizacion_y_contratos' and its files: 'E1.cpp', 'E2.cpp', 'E3.cpp', and 'E4.cpp'. The status bar at the bottom indicates 'Linea: 1, Carácter: 1' and 'SPC LF UTF-8'.

```

Ejercicio 5 (S = 15), rango valido [-15, 30]

Escenario 1
Descartadas (fuera de [-15, 30]): -20 40
Validas: 6 de 8
Promedio: 6.33333
Minimo valido: -15
Maximo valido: 30

Escenario 2
Descartadas (fuera de [-15, 30]): -16 -17 31 80
Validas: 0 de 4
Sin mediciones validas: promedio: ninguna medicion cae dentro del rango

Modificacion final
Nuevo rango valido [-15, 45]
Descartadas (fuera de [-15, 45]): -20
Validas: 7 de 8
Promedio: 11.1429
Minimo valido: -15
Maximo valido: 40

Elija un ejercicio:
1. Ejercicio 1 - Radiografia de un main monolitico
2. Ejercicio 2 - Laboratorio de paso de parametros
3. Ejercicio 3 - Tres archivos y dos clases de error
4. Ejercicio 4 - Pruebas antes de confiar
5. Ejercicio 5 - Mini-proyecto: monitor de mediciones
6. Salir
Opcion: |
  
```

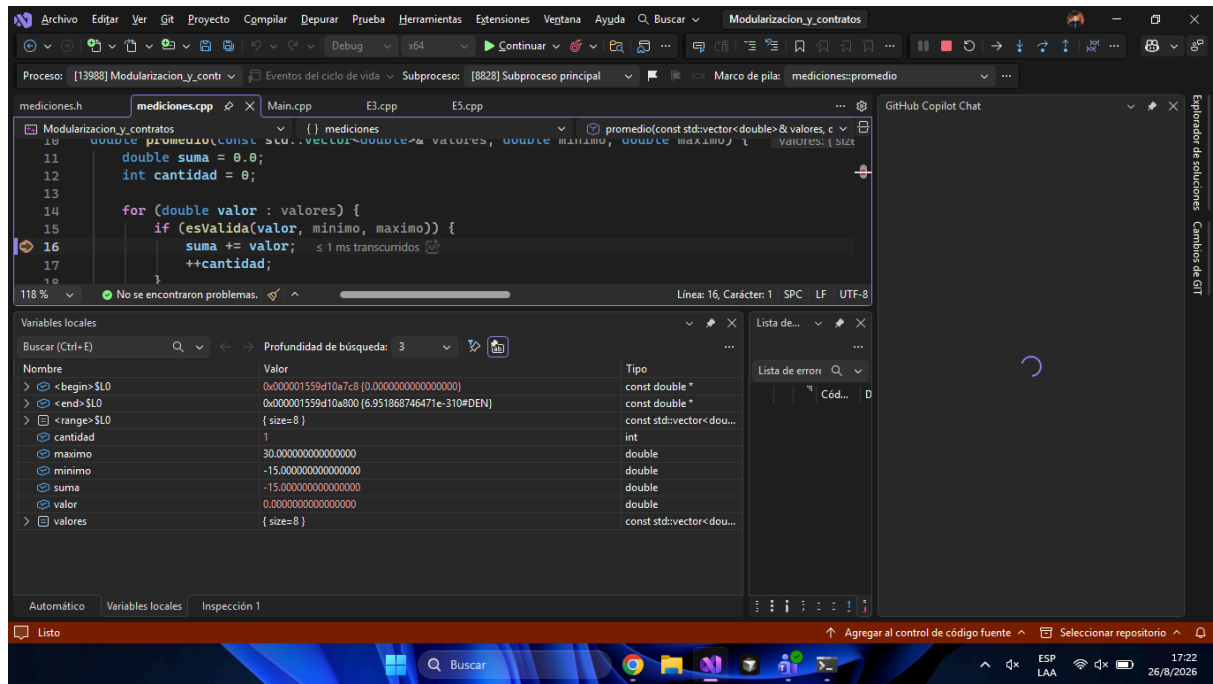
6)

```

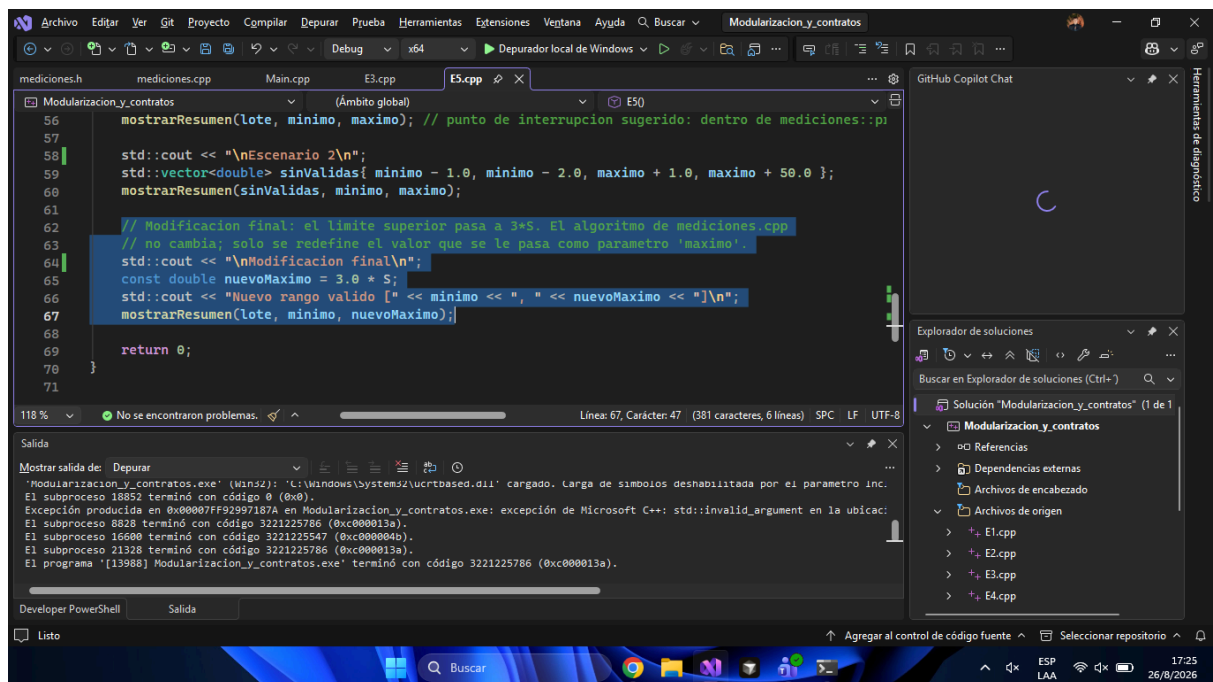
double promedio(const std::vector<double>& valores, double minimo, double maximo) {
    double suma = 0.0;
    int cantidad = 0;

    for (double valor : valores) {
        if (esValida(valor, minimo, maximo)) {
            suma += valor;
            ++cantidad;
        }
    }
}
  
```

Nombre	Valor	Tipo
<begin>\$L0	0x0000001416010a7c0 (-15.000000000000000000)	const double *
<end>\$L0	0x0000001416010a800 (6.951868746471e-310#DEN)	const double *
<range>\$L0	{ size=8 }	const std::vector<double,std::all...
cantidad	0	int
maximo	30.000000000000000000	double
minimo	-15.000000000000000000	double
suma	0.000000000000000000	double
valor	-15.000000000000000000	double
valores	{ size=8 }	const std::vector<double,std::all...



7)



Para subir el límite superior a $3 \times S$ no modifiqué ninguna función de mediciones.cpp: promedio, contarValidas, minimoValida y maximoValida siguen recibiendo el máximo como parámetro, tal como estaban diseñadas. El único cambio fue agregar la constante `nuevoMaximo = 3.0 * S` en E5.cpp y pasarla como argumento en la nueva llamada a `mostrarResumen`. Esto demuestra que separar la política de validación (el rango) de la lógica de cálculo permite ajustar reglas de negocio sin tocar el algoritmo: con el rango `[-15, 45]`, el valor 40 ahora entra, quedando 7 de 8 mediciones válidas en vez de 6.

Anexo de IA

#	Num ejercicio	Prompt	Qué tomé	Qué no tomé
1	1	¿Cómo separo este main grande en funciones más chicas?	La necesidad de modularizar el programa.	Explicaciones teóricas sin código.
2	1	¿Qué parte del código conviene poner en cada archivo?	La organización por módulos.	Código concreto, porque en E1 no hay implementación.
3	1	¿Cómo quedaría el main si lo simplifico para llamar a funciones?	La idea de delegar trabajo a otras funciones.	Lógica embebida en un solo bloque.
4	2	¿Qué pasa si paso un vector<int> por valor y lo modifico adentro?	La copia del vector y que no cambia el original.	Tipos distintos a vector<int>.

5	2	¿Cómo cambio una función para que reciba el vector sin copiarlo?	El uso de referencia.	Copias innecesarias.
6	2	¿Cómo hago una función que lea un vector sin poder modificarlo?	El uso de const vector<int>&.	Cambios sobre los datos.
7	2	¿Cómo manejo el caso en que un vector está vacío antes de usar front()?	La validación del vector vacío.	Acceso inseguro sin chequeo.
8	2	¿Cómo hago una función que recorra el vector y aumente todos los valores?	La lógica de incrementar.	Dejar el vector sin cambios.
9	3	¿Cómo filtro un vector para quedarme solo con los valores dentro del rango?	La validación por rango.	Valores fuera del intervalo.
10	3	¿Cómo calculo el promedio solo con los elementos válidos?	El promedio de los datos filtrados.	Incluir inválidos en el cálculo.
11	3	¿Cómo uso S para definir el rango válido del ejercicio?	La semilla personal en el código.	Un rango fijo sin depender de S.
12	4	¿Cómo pruebo que enRango acepta también los límites?	La validación inclusiva de extremos.	Comparaciones mal puestas.
13	4	¿Cómo pruebo que enRango devuelve false para valores fuera del intervalo?	Los casos por debajo y por arriba.	Tratar todo como válido.
14	4	¿Cómo verifico que promedioValidos lance excepción si no hay datos válidos?	El manejo de std::invalid_argument.	Devolver un número cualquiera.
15	5	¿Cómo armo un resumen de mediciones mostrando válidas, inválidas y estadísticas?	El uso de varias funciones del módulo.	Mezclar lógica sin validar.
16	5	¿Cómo hago para que el programa siga funcionando aunque un lote no tenga válidos?	El bloque try/catch.	Caerse sin manejar el error.
17	5	¿Cómo cambio solo el máximo permitido sin tocar la lógica de cálculo?	Pasar otro parámetro como nuevoMaximo.	Reescribir todo el algoritmo.